



Блог

# AI SDK 7



Грегор Мартинус, Ларс Граммел, Феликс Арнтц, Аюш Капур, Джош Сингх

25 июня 2026 г.

9 минут чтения

Копировать

AI SDK, который загружается более 16 миллионов раз в неделю, представляет собой TypeScript SDK для создания приложений искусственного интеллекта, функций, фреймворков и агентов для любого поставщика моделей. Это тот же уровень, на котором построен eve, фреймворк агента с открытым исходным кодом Vercel.

AI SDK 7 расширяет возможности работы с агентами в пяти областях:

- **Разработка агентов** с контролем рассуждений, контекстом инструмента и среды выполнения, поддержкой файлов провайдеров и навыков, приложениями MCP и пользовательским интерфейсом терминала.
- **Запуск агентов** с подтверждением инструмента, поддержкой отказоустойчивости (`WorkflowAgent`), таймаутов и песочницы.
- **Интеграция любого набора инструментов для работы с агентами**, например Codex, Claude Code, Deep Agents, OpenCode или Pi.
- **Наблюдение за агентами** с помощью телеметрии, канала трассировки Node.js, событий жизненного цикла и статистики производительности.
- **Выйдите за рамки текстовых агентов** с помощью голосовой поддержки в реальном времени и генерации видео, не зависящих от поставщика.

```
pnpm add ai@latest
```

Установить AI SDK 7



Обновляйтесь с AI SDK 6? Запустите `npx @ai-sdk/codemod v7` для автоматического перехода с минимальными изменениями кода или воспользуйтесь навыком миграции:

```
npx skills add vercel/ai --skill migrate-ai-sdk-v6-to-v7
```

## Разработка агентов

Для создания хорошо функционирующих агентов требуется детальный контроль над логическими выводами модели, контекстом инструмента и обработкой файлов.

## Управление логическими выводами

Большинство передовых моделей поддерживают настраиваемые логические выводы, но каждый API-провайдер предоставляет их по-своему.

В AI SDK 7 это стандартизировано с помощью опции `reasoning` для `generateText` и `streamText`. Она соответствует настройкам логических выводов, используемым провайдером, и позволяет контролировать трудозатраты на логические выводы с помощью одной строки. Вы также можете вернуться к настройкам провайдера, если вам нужна более подробная конфигурация рассуждений для конкретного провайдера.

TS agent.ts

```
1 import { generateText } from 'ai';
2 const result = await generateText({
3   model,
4   prompt,
5   reasoning: 'high',
6 });
```

Настройка усилий при рассуждении с помощью одного параметра

Подробнее в [документации по рассуждениям](#).

## Контекст инструмента

Инструменты все чаще разрабатываются независимо от конкретных агентов или приложений. Например, сторонние компании предлагают инструменты, которые позволяют агентам использовать их API. Поэтому инструментам требуются дополнительные входные данные, которые не генерируются большими языковыми моделями, например ключи API или параметры конфигурации.

В AI SDK 7 добавлен полностью типизированный контекст инструмента, который можно указать для каждого инструмента с помощью схемы. Контекст ограничен инструментом, чтобы сторонние инструменты не могли получить доступ к ненужному им контексту.

TS agent.ts

```

1  const agent = new ToolLoopAgent({
2    model,
3    tools: {
4      weather: tool({
5        description,
6        inputSchema,
7        contextSchema: z.object({
8          apiKey: z.string(),
9        }),
10     execute: async (input, { context: { apiKey } }) => {
11       // ...
12     },
13   }),
14 },
15   toolsContext: {
16     weather: { apiKey: process.env.WEATHER_API_KEY! },
17   },
18 });

```

Scoping an API key to the tool that needs it

Learn more about [Tool Context](#)

## Runtime context

For more complex agentic loops, you often need variables that you can access and modify in `prepareStep` to adjust prompts, model selection, and more.

AI SDK 7 introduces a typed runtime context available during step preparation and tool approval functions, with optional telemetry support. This enables you to encapsulate more logic in `ToolLoopAgent` and share those agents with that internal logic.

 agent.ts



```

1  const agent = new ToolLoopAgent({
2    // setup runtime context
3    runtimeContext: {
4      var1: "something",
5    },
6    prepareStep: async ({ runtimeContext, steps }) => {
7      // use runtime context
8      // return updated runtime context
9    },
10 });

```

Learn more about [Runtime Context](#).

## Provider file uploads

Many agent workflows require handling large inputs, such as PDFs, images, datasets, or other artifacts. Sending those files inline is slow and wasteful, especially for stateless inference, where they get sent over and over again.

AI SDK 7 adds a top-level `uploadFile` API that lets you upload a file once and then pass a lightweight reference into subsequent model calls. This avoids re-uploading the same bytes repeatedly, making inference faster and saving bandwidth during repeated or multi-step runs.

`uploadFile` can be used with any providers that offer a file uploading endpoint. The function returns a provider reference object that is portable across providers.

TS upload.ts



```
1  const { providerReference } = await uploadFile({
2    api: openai.files(),
3    data: readFileSync('./photo.png'),
4    filename: 'photo.png',
5  });
6  const result = await streamText({
7    model: openai.responses('gpt-5.5'),
8    messages: [
9      {
10       role: 'user',
11       content: [
12         { type: 'text', text: 'Describe what you see in this image.' },
13         { type: 'file', mediaType: 'image', data: providerReference },
14       ],
15     },
16   ],
17 });
```

Upload a file once, pass a reference into subsequent model calls

Learn more about [Provider File Uploads](#)

## Provider skill uploads

Sending skills inline on every request to provider-managed container environments has the same overhead problem as sending files inline.

AI SDK 7 adds a top-level `uploadSkill` API that lets you upload a skill once and then use a reference to it in subsequent inference calls. Similar to `uploadFile`, the function returns a provider reference object.

TS upload.ts



```
1  const { providerReference } = await uploadSkill({
2    api: anthropic.skills(),
3    files: [
4      {
5        path: 'my-skill/SKILL.md',
6        content: readFileSync('./SKILL.md'),
7      },
8    ],
9    displayTitle: 'My Skill',
10  });
11  const result = await streamText({
12    model: anthropic('claude-sonnet-4-6'),
13    tools: {
14      code_execution: anthropic.tools.codeExecution_20260120(),
15    },
16    prompt: 'Use the my-skill skill to complete the task.',
17    providerOptions: {
18      anthropic: {
19        container: {
20          skills: [{ type: 'custom', providerReference }],
21        },
22      } satisfies AnthropicLanguageModelOptions,
23    },
24  });
```

Upload a skill once, reference it across inference calls

Learn more about [Provider Skill Uploads](#).

## MCP Apps

MCP has become a common way to connect agents to tools and resources. But not every tool should be model-visible, and some MCP servers need to expose specialized UI alongside their tools.

AI SDK 7 adds support for MCP Apps. MCP servers can now separate model-visible tools from app-only tools, preserve app metadata, and render app UIs inside sandboxed iframes. A JSON-RPC bridge connects tools, resources, and display interactions.

This lets you build richer agent experiences where the model can use the tools it needs, while the user sees an app-specific interface for review, configuration, or interaction.

## MCP Apps in AI SDK

h

An MCP App rendering its UI alongside the agent

 components/chat.tsx



```
1 import { experimental_MCPAppRenderer as MCPAppRenderer } from '@ai-sdk/react';
2 import { isToolUIPart } from 'ai';
3 {
4   messages.map(message =>
5     message.parts.map(part =>
6       isToolUIPart(part) ? (
7         <MCPAppRenderer
8           key={part.toolCallId}
```

```

9      part={part}
10     sandbox={{ url: '/mcp-app-sandbox', className: 'h-96 w-full' }}
11     loadResource={app => fetch(`/api/mcp-apps?uri=${app.resourceUri}`)}
12     handlers={{ allowedTools: ['refreshDashboard'] }}
13   />
14   ) : null,
15   ),
16   );
17 }

```

Rendering MCP app UIs alongside model output

Start building your first [MCP App with AI SDK](#) today.

## TUI

When developing agents, you need to be able to quickly test them without writing a full app. AI SDK 7 adds a terminal UI (TUI) package that lets you run agents with just a few lines of code:

The TUI is interactive, supports reasoning and tools, and renders markdown as formatted text.



An agent running in the terminal UI

 dev.ts



```

1  import { runAgentTUI } from '@ai-sdk/tui';
2  await runAgentTUI({ agent });

```

Learn more about [creating your own terminal agent](#).

## Run agents

As agents become more autonomous and longer running, the need for approvals, durability, sandboxing, and robustness increases.

### Tool approvals

AI SDK 7 supports agent-level tool approvals that can be automatic or involve a human in the loop, with these approval types:

- Simple `user-approval` for particular tools.
- Tool approval function for a particular tool that can auto-approve, auto-deny, or forward to user approval.
- Generic catch-all tool approval functions.

Tool approvals are defined on `ToolLoopAgent`, `generateText`, and `streamText`, because the usage scenario of a particular tool drives the need for approvals.

TS agent.ts



```
1  const agent = new ToolLoopAgent({  
2    model,  
3    tools: { weather: weatherTool },  
4    toolApproval: {  
5      weather: 'user-approval',  
6    },  
7  });
```

Requiring user approval before a tool executes

For higher-risk workflows, AI SDK 7 introduces opt-in HMAC-signed tool approvals to prevent forged approvals. The SDK also hardens replay behavior by revalidating tool inputs and policies before continuing execution.

See [how tool approvals work](#).

## WorkflowAgent (Durability)

When an agent run spans multiple steps or waits for a human approval, a process restart or deployment in the middle of that run means starting over. AI SDK 7 introduces `@ai-sdk/workflow` and `WorkflowAgent` for durable, resumable agent execution that survives process restarts, deploys, interruptions, and delayed approvals.

`WorkflowAgent` supports workflow-based streaming, tools, approvals, callbacks, `prepareCall`, and provider model serialization across workflow step boundaries. It also supports typed runtime context for shared agent state and stable telemetry.

Callbacks now include richer execution data such as step numbers, previous results, duration, and success or failure information. Invalid tool calls are preserved without executing invalid tools, and tool `toModelOutput` conversion can preserve raw outputs for UI and callbacks.

Learn how to [build an agent with WorkflowAgent](#).

## Timeouts

Agents can stall in more ways than a simple request can: a provider can open a stream and stop sending chunks, a tool can hang, or a multi-step run can exceed its total budget.

AI SDK 7 adds first-class timeout configuration across text generation and agent APIs, including total, per-step, per-chunk, and per-tool limits. Timeout aborts use `TimeoutError`, and abort reasons propagate through stream and UI protocols.

TS agent.ts



```
1  const result = await generateText({
2    model,
3    tools: { weather: weatherTool, slowApi: slowApiTool },
4    timeout: {
5      totalMs: 60000, // 60 seconds total
6      stepMs: 10000, // 10 seconds per step
7      chunkMs: 2000, // abort if no chunk received for 2 seconds
8      toolMs: 5000, // default for all tools
9      tools: {
10        weatherMs: 3000, // 3 seconds for weather tool
11        slowApiMs: 10000, // 10 seconds for slow API tool
12      },
13    },
14    prompt: 'What is the weather in San Francisco?',
15  });
```

Learn more about [timeouts](#).

## Sandbox support

Agents that run shell commands, read and write files, or execute generated code need a consistent execution environment, but the underlying sandbox often changes across local dev, CI, and production. AI SDK 7 adds a first-class `SandboxSession` abstraction for portable command execution in tools and agents. Tools can be developed independently of any particular sandbox, and you can use any sandbox-aware tool with any sandbox provider.

Sandboxed environments, such as [Vercel Sandbox](#), are ideal for this purpose.

## Integrate any agent harness

Agent runtimes are moving beyond a single application server. Teams want to run the same agent logic inside coding environments, hosted sandboxes, local sessions, and third-party harnesses.

## HarnessAgent

AI SDK 7 introduces experimental harness abstractions and `HarnessAgent`: one API to run fully configured, established agent harnesses such as Claude Code, Codex, and Pi. Harnesses are configurable with a sandbox to operate in, custom instructions, skills, and tools. Run established harnesses through a consistent interface, configure each one independently, and swap one out without changing your integration layer.

Under the hood, the abstraction consists of a v1 adapter spec, bridge support, and expanded sandbox session primitives for creating and resuming sessions. Harness sessions can be parked and resumed, and even individual turns can be interrupted and resumed mid-flight.

`HarnessAgent` implements AI SDK's `Agent` interface, so its `generate` and `stream` return values are fully compatible with existing AI SDK integrations, and `useChat()` and the new TUI work without any additional wiring.

 agent.ts



```
1  const agent = new HarnessAgent({  
2    harness: claudeCode,
```

```

3   sandbox: createVercelSandbox({
4     runtime: 'node24',
5     ports: [4000],
6   }),
7   instructions:
8     'You are a careful coding assistant. Prefer small changes and explain tradeoffs.',
9   skills: [
10    {
11      name: 'review-github-pr',
12      description: 'Review a GitHub pull request. Use when asked to review a pull request',
13      content:
14        'Use the `readGitHubPullRequest` tool to fetch the context about the relevant pull request.',
15        'If the pull request refers to an issue, fetch the relevant issue context as well.',
16    },
17  ],
18  tools: { readGitHubIssue, readGitHubPullRequest },
19 });

```

Configuring Claude Code as a HarnessAgent with a sandbox and custom skills

Learn more about [AI SDK Harnesses](#).

## Observe agents

Understanding how your agents behave in production is challenging. AI SDK 7 makes observability a first-class part of building agents.

## Telemetry

AI SDK 7 revamps telemetry around a single, extensible integration system. Instead of wiring lifecycle callbacks into every `generateText` or `streamText` call, register telemetry once at application startup:

 instrumentation.ts



```

1  import { registerTelemetry, generateText } from 'ai';
2  import { OpenTelemetry } from '@ai-sdk/otel';
3  registerTelemetry(new OpenTelemetry());
4  const result = await generateText({
5    model: "google/gemini-3.5-flash",
6    prompt: 'Write a short story about a cat.',
7    telemetry: {
8      functionId: `story-agent`,
9    },

```

```
10  });
```

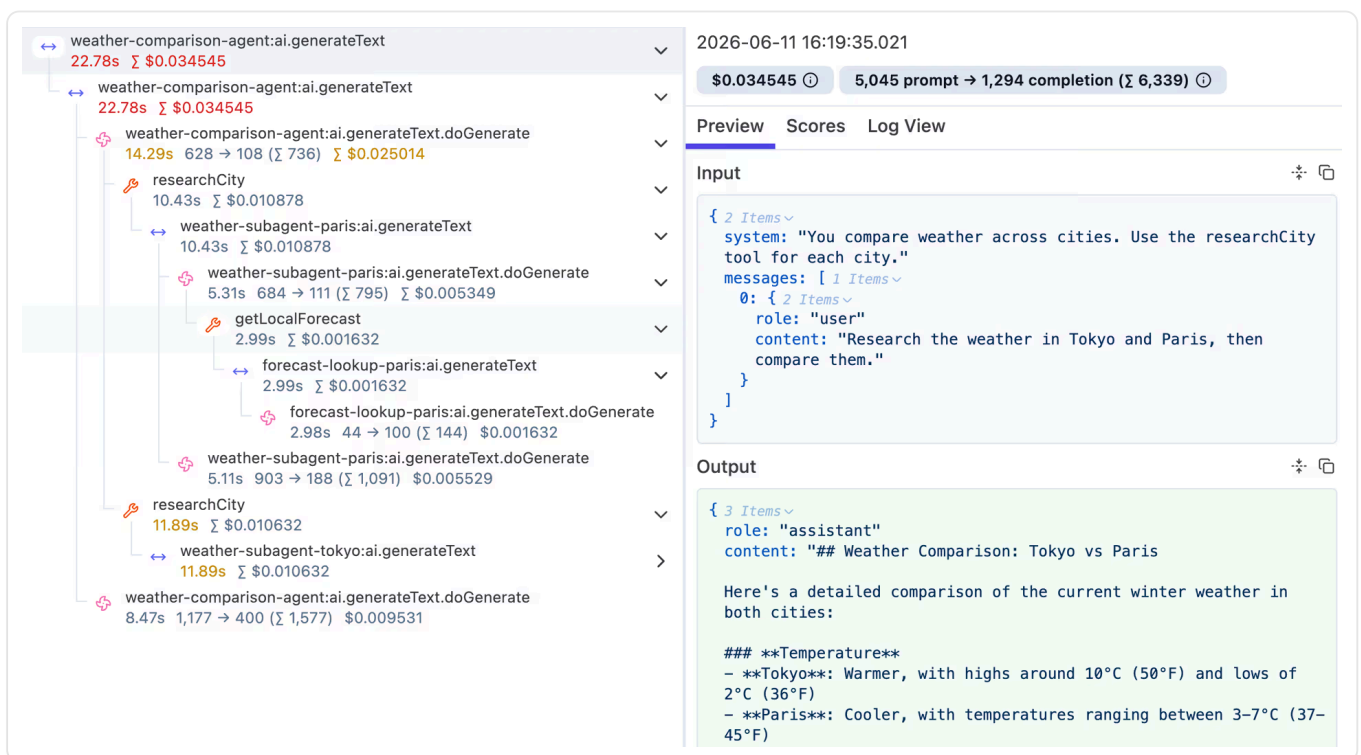
Registering OpenTelemetry once at application startup

The redesign includes:

- Dedicated telemetry interfaces for 3rd-party provider integration
- Global coverage of all AI SDK functions with a single registration
- Optional OpenTelemetry integration using the latest GenAI semantic conventions
- Node.js tracing channel support

Observability integrations: Datadog, Langfuse, Braintrust, Raindrop, Sentry, Laminar, Langsmith.

Traces now capture the full shape of an AI operation, including the root generation, each model call, individual steps, tool executions, embeddings, reranking, usage, errors, and selected runtime or tool context.



An observed trace of an agent in Langfuse with multiple steps and tool calls

You can find more details in the [AI SDK Telemetry documentation](#).

## Node.js tracing channel

AI SDK 7 adds support for Node.js tracing channels via `node:diagnostics_channel`. The SDK emits structured telemetry events on the `ai:telemetry` channel for `generateText`, `streamText`, model calls, tool executions, embeddings, and reranking.

An observability provider can subscribe once via its instrumentation package and automatically convert AI SDK activity into traces, preserving async context across streaming responses and tool calls.

instrumentation.ts

```
1 import { tracingChannel } from 'node:diagnostics_channel';
2 import {
3   AI_SDK_TELEMETRY_TRACING_CHANNEL,
4   type TelemetryTracingChannelMessage,
5 } from 'ai';
6 tracingChannel(AI_SDK_TELEMETRY_TRACING_CHANNEL).subscribe({
7   start(message) {
8     const { type, event } = message as TelemetryTracingChannelMessage;
9     console.log(`AI SDK ${type} started`, event);
10  },
11  asyncEnd(message) {
12    const { type } = message as TelemetryTracingChannelMessage;
13    console.log(`AI SDK ${type} completed`);
14  },
15 });
```

Subscribing to AI SDK telemetry events via the Node.js tracing channel

You can learn more in the [tracing channel documentation](#).

## Performance statistics

AI SDK 7 adds per-step performance statistics for model output, streaming behavior, and tool execution. You can answer questions like: How long did it take the model to start responding? How fast did tokens arrive? Which tool took the most time?

app.ts

```
1 import { streamText } from 'ai';
2 const result = streamText({
3   model: 'openai/gpt-5',
4   prompt: 'Write a short product announcement.',
5 });
6 for await (const chunk of result.textStream) {
```

```

7   process.stdout.write(chunk);
8 }
9 const { performance } = await result.finalStep;
10 console.log({
11   responseTimeMs: performance.responseTimeMs,
12   outputTokensPerSecond: performance.outputTokensPerSecond,
13   timeToFirstOutputMs: performance.timeToFirstOutputMs,
14 });

```

Getting per-step latency and throughput from a streamed response

Learn more about [performance statistics](#).

## Lifecycle events

Production agents need lifecycle hooks because recording state, billing, and debugging all depend on knowing exactly when runs, steps, and tools start and finish. AI SDK 7 makes callbacks fire consistently across model calls, agents, tools, and other functions, so you can observe when each started, which model ran, how many tokens it used, and how it finished.

 agent.ts



```

1  import { generateText } from 'ai';
2  const result = await generateText({
3    model: 'openai/gpt-5',
4    prompt: 'What is the meaning of life',
5    runtimeContext: {
6      userId: 'user_123',
7      feature: 'launch-copy',
8    },
9    onStart({ callId, modelId, runtimeContext }) {
10      console.log('Request started', {
11        callId,
12        modelId,
13        userId: runtimeContext.userId,
14      });
15    },
16    onEnd({ callId, usage, finishReason }) {
17      console.log('Request finished', {
18        callId,
19        finishReason,
20        totalTokens: usage.totalTokens,
21      });
22    },
23  });

```

You can find more details in the [Lifecycle Callbacks documentation](#).

## Provider-agnostic realtime support

Realtime model APIs are powerful, but each provider exposes sessions, audio, tools, and browser authentication differently.

AI SDK 7 adds experimental provider-agnostic realtime support for direct browser WebSocket sessions. The SDK supports server-created ephemeral tokens, provider implementations for OpenAI, Google, and xAI, and a React realtime hook that returns `UIMessage[]`.

Realtime sessions support audio transcription and client-driven tool calling, so you can build voice agents, collaborative copilots, and low-latency interactive interfaces without binding your UI to one provider's event format.

AI Gateway also supports normalized realtime sessions through `gateway.experimental_realtime()`, including WebSocket subprotocol auth, model query selection, and validated provider options.

 components/realtime.tsx



```
1  const realtime = experimental_useRealtime({
2    model: gateway.experimental_realtime('openai/gpt-realtime-2'),
3    api: {
4      token: '/api/realtime/setup',
5    },
6    onToolCall: async ({ toolCall }) => {
7      // handle client side or sent server requests
8    },
9  });
```

Connecting to a realtime session from the browser

Learn more about [realtime](#).

## Video generation

AI applications are expanding beyond text and images. AI SDK 7 introduces experimental `generateVideo` support with provider implementations for fal, Google AI Studio, Google Vertex, and Replicate.

Video generation in AI SDK 7 uses video-specific model resolution, supports string-based model lookup through the default provider, and includes safer bounded download handling with configurable size limits and abort support.

TS app.ts

```
1 import { experimental_generateVideo as generateVideo } from 'ai';
2 const { videos } = await generateVideo({
3   model: "google/veo-3.1-generate-001",
4   prompt: 'A cat walking on a treadmill',
5   aspectRatio: '16:9',
6 });
```

Generating a video with a single API call

Learn more about [generating video](#).

## Getting started

Install AI SDK 7 with one command.

```
pnpm add ai@latest
```

Install AI SDK 7

- [Get started with AI SDK 7](#)
- [Check out the full changelog](#)
- [Migrate to v7](#)
- [Join the community](#)

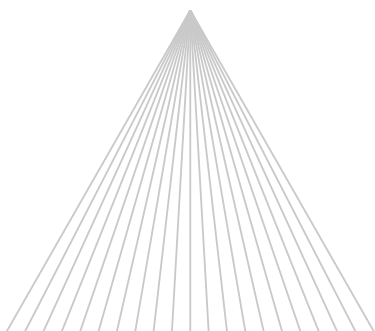
## Contributors

AI SDK 7 is the result of the combined work of our core team at Vercel (Gregor, Lars, Felix, Aayush, Josh, Nico) and our amazing community of contributors:

[Oxr3ngar](#), [31Carlton7](#), [A-Vamshi](#), [Abdulwadood-zawity](#), [abhicris](#), [adithya-tako](#), [AhmadYasser1](#), [ahmedrowaihi](#), [allenzhou101](#), [anaclumos](#), [arnaugomez](#), [auscaster](#), [AVtheking](#), [B-Step62](#), [bb220](#), [ben-vargas](#), [benyebai](#), [bittere](#), [blurrah](#), [bolaabanjo](#), [boylec](#), [BrianHung](#), [BrianP8701](#), [chenxin-yan](#), [christian-bromann](#), [Christian-Sidak](#), [cipher416](#), [CloudFaye](#), [codewarnab](#), [codicecustode](#), [codylittle](#), [cristiandrei1234](#), [csidak](#), [ctate](#), [cyphercodes](#), [defrex](#), [dflynn15](#), [dinmukhamedm](#), [dnukumamras](#), [DongSeonYoo](#), [dukex](#), [edawerd](#), [Edwardlrby](#), [edwardwc](#), [ellis-driscoll](#), [embedder-dev](#), [etairl](#), [EurFelux](#), [eyueldk](#), [fahe1em1](#), [Falven](#), [fran3cc](#), [gdborton](#), [genmin](#), [geraint0923](#), [Ghitahour](#), [GidianB](#), [grant0417](#), [gsdv](#), [guillemwilly](#), [hank9999](#), [harpreetarora](#), [haydenbleasel](#), [he-yufeng](#), [heiwen](#), [hkd987](#), [hntrl](#), [http-samc](#), [i5d6](#), [ismaelrumzan](#), [Jaaneek](#), [jaderiverstokes](#), [jakobhoeg](#), [Jaksenc](#), [jarod](#), [jaydeep-pipaliya](#), [jeremyphilemon](#), [jerilynzheng](#), [jerome-benoit](#), [jferrettiboke](#), [jlsandri](#), [JoanLaRosa](#), [joapedroassad](#), [JohnnyHBon](#), [josh-williams](#), [jovanwongzixi](#), [JulesGuesnon](#), [Kage18](#), [kagura-agent](#), [kairosai](#), [kaizen403](#), [karthik-idikuda](#), [kimchinn](#), [kkawamu1](#), [kkdawkins](#), [leog25](#), [leothorp](#), [liaoliaojun](#), [lihuimingxs](#), [Mahendradeokar](#), [MarcACard](#), [marcusschiesser](#), [markmcd](#), [max-programming](#), [MaxwellCalkin](#), [mclenhard](#), [MehediH](#), [Melkeydev](#), [michael-han-dev](#), [michaelcummings12](#), [Mmartinrusso](#), [monadoid](#), [montyanderson](#), [more-by-more](#), [mrpaaradox](#), [msullivan](#), [muniter](#), [muraliavarma](#), [murataslan1](#), [mvanhorn](#), [myprototypewhat](#), [Nezz](#), [nicoloboschi](#), [nielskaspers](#), [Nutlope](#), [nwalters512](#), [ohansFavour](#), [ousugo](#), [pablof7z](#), [Pash10g](#), [patrikdevlin](#), [pauelliotco](#), [pavel-y-ivanov](#), [PierreLeGuen](#), [posva](#), [Pranav-Wadhwa](#), [privatenumbr](#), [quuu](#), [R-Taneja](#), [raphaeleidus](#), [reynkonig](#), [Ricardo-M-L](#), [richardsolomou](#), [robechun](#), [rubdos](#), [samjbobb](#), [SamyPesse](#), [seojcarlos](#), [shaper](#), [shrey150](#), [shubham-021](#), [shujanislam](#), [ShyamSathish005](#), [sleitor](#), [Subr1ata](#), [syedddhasnainn](#), [sylviezhang37](#), [szymonrybczak](#), [t-mdo](#), [techwraith](#), [theQuert](#), [thestonechat](#), [timvucina-soniox](#), [tomdale](#), [tresorama](#), [tsuzaki430](#), [turisanapo](#), [Und3rf10w](#), [undo76](#), [ushiromiya-lion](#), [visyat](#), [wong2](#), [Xiang-CH](#), [xlianghang](#), [zapagenrevdale](#), [Zawwarsami16](#), [zirkelc](#), [zxuhan](#).

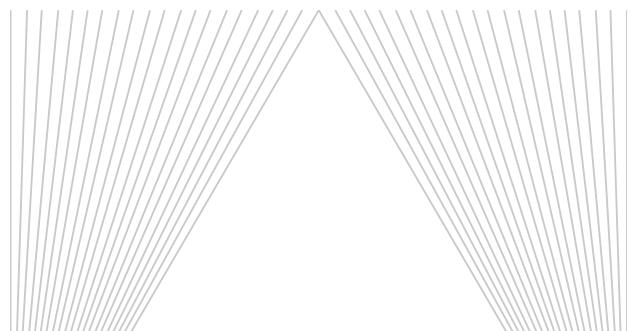
Your feedback, bug reports, and pull requests on GitHub have been instrumental in shaping this release. We're excited to see what you'll build with these new capabilities!

## Explore



17 Jun 2026

**Introducing eve**



17 Jun 2026

**The Agent Stack**

# Ready to deploy?

Start Deploying

Talk to an Expert

## Agent Stack

AI SDK

AI Gateway

Sandbox

Workflows

Connect NEW

Passport NEW

eve NEW

## Core Platform

CI/CD

Content Delivery

Fluid Compute

Observability

## Security

Platform Security

WAF

Bot Management

Bot ID

## Tools

Vercel Drop NEW

Vercel Agent

Vercel Plugin NEW

Agent Skills

Next.js

Domains

v0

## Frameworks

eve NEW

Nuxt

SvelteKit

Nitro

Turborepo

Tanstack Start

FastAPI

xmcp

All frameworks

## SDKs

Vercel SDK

Workflow SDK NEW

Flags SDK

Chat SDK NEW

Queues SDK NEW

Streamdown

## Build

AI Apps

Web Apps

Marketing Sites

Platforms

Commerce

## Learn

Docs

Blog

Changelog

Knowledge Base

Academy

## Explore

Customers

Marketplace

Templates

Partner Finder

Vercel + AWS

Platform Engineers  
Design Engineers

Articles  
Community

Company

About  
Careers  
Press  
Events  
Startups  
Shipped on Vercel  
Open Source Program  
Enterprise  
Pricing  
Help



Legal & Trust

Privacy Policy  
Terms of Service  
Cookie Policy  
DPA  
Acceptable Use Policy  
Legal (all documents)  
Trust Center  
Status

Social

GitHub  
X  
LinkedIn  
YouTube  
Instagram

LOADING STATUS...

